

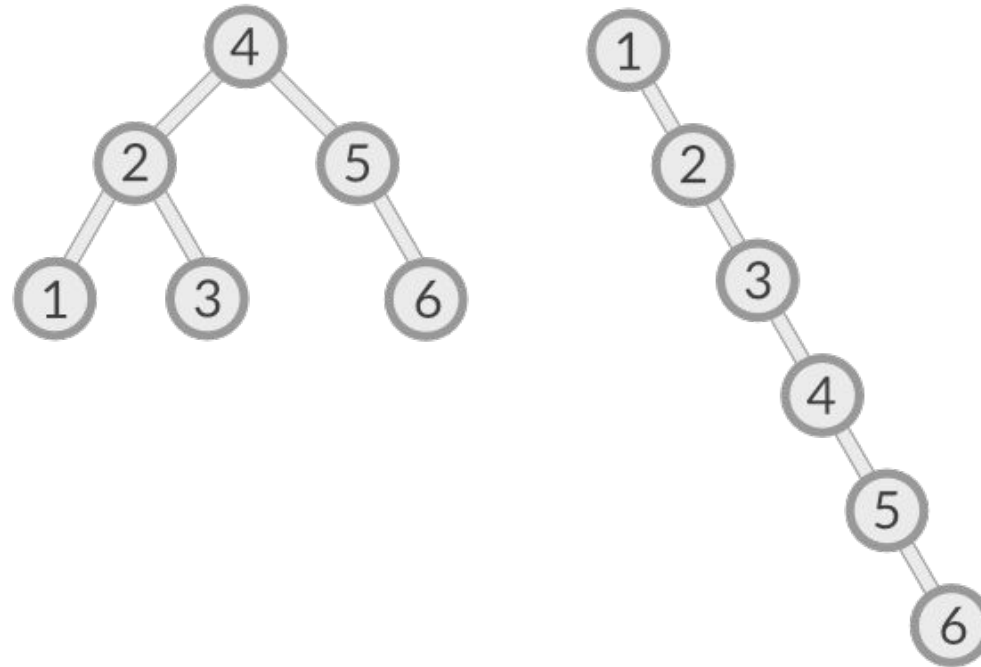
Introduction to Algorithms

Levon Prazyan

Course Overview

- Introduction
- Sorting and Searching
- Arrays and Lists
- Stacks and Queues, Double Ended Queue
- Heaps
- **Trees**
- Hash Tables
- Dynamic Programming
- Greedy Algorithms
- Graph Algorithms
- Network Flow
- NP-Completeness

Balanced vs Unbalanced



- Balanced BSTs guarantee that basic dynamic-set operations, such as *SEARCH*, *PREDECESSOR*, *SUCCESSOR*, *MINIMUM*, *MAXIMUM*, *INSERT*, and *DELETE* take $O(\log n)$ time in the worst case.

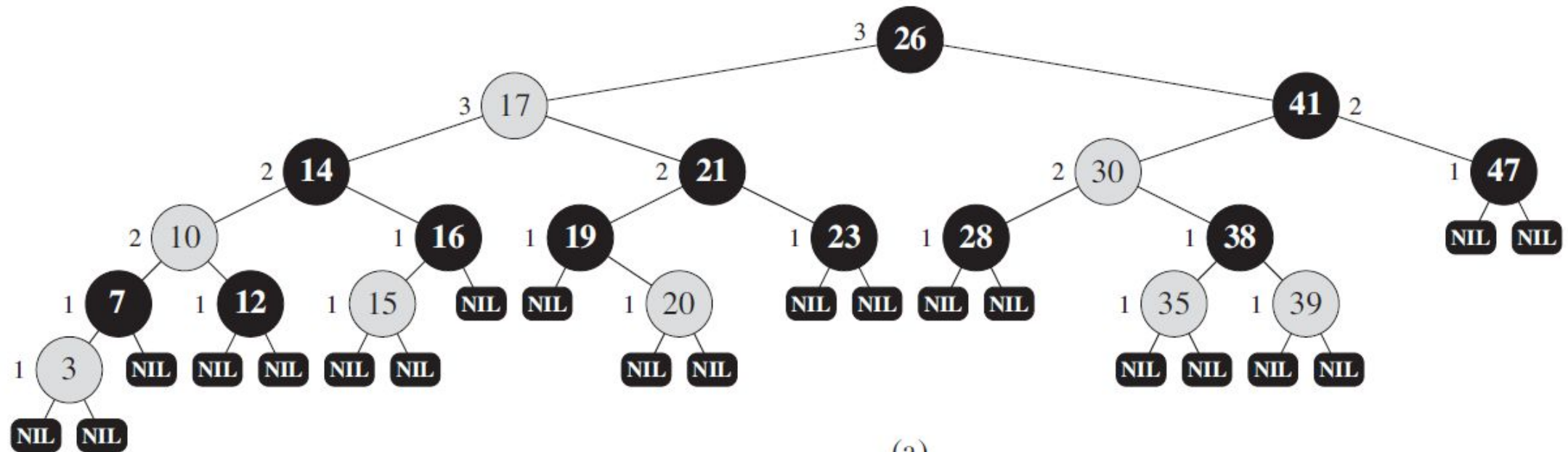
Red-Black Tree: Definition

A Red-Black Tree is a type of self-balancing binary search tree with one extra bit of storage per node: its color, which can be either RED or BLACK.

The tree maintains the following invariants:

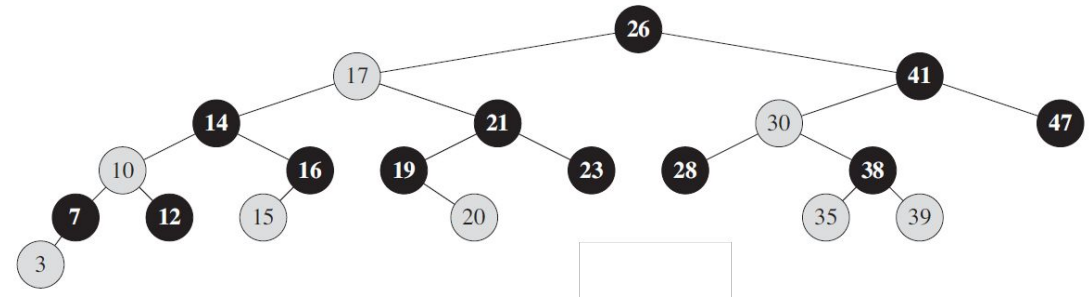
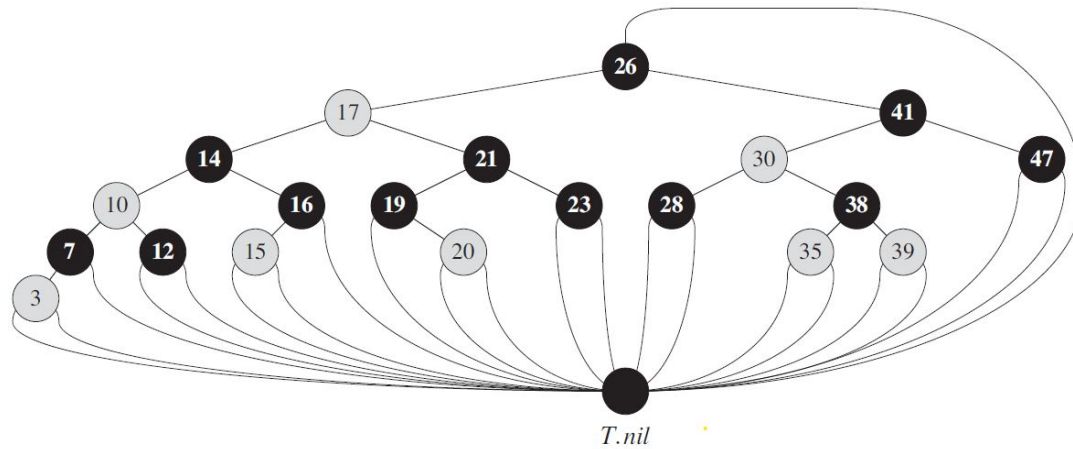
1. Every node is either red or black.
2. The root is always black.
3. All leaves (NIL nodes) are black.
4. Red nodes cannot have red children (i.e., no two red nodes can be adjacent).
5. Every path from a node to its descendant NIL nodes must have the same number of black nodes.

Red-Black Tree: Example (1)



(a)

Red-Black Tree: Example (2)



Red-Black Tree: Black-Height

Black-height of the node is the number of black nodes (not counting the node itself) on any path from that node down to a leaf (NIL): bh

$$bh(NIL) = 0$$

Black-height of a tree is a black-height of its root.

Lemma: The height h of a red-black tree T is at most $2 * bh(T.root)$.

Proof: No two red nodes in a row. Any path from root to leaf, red nodes must be followed by black nodes. The longest possible path

$$\begin{aligned} T.root &\rightarrow red \rightarrow black \rightarrow red \rightarrow \dots \rightarrow NIL \\ h &\leq \max(count(black) + count(red)) = 2 * count(black) \\ h &\leq bh(T.root) \end{aligned}$$

Red-Black Tree: Height

Lemma: A red-black tree with n internal nodes has height at most $2 \log(n + 1)$

Proof:

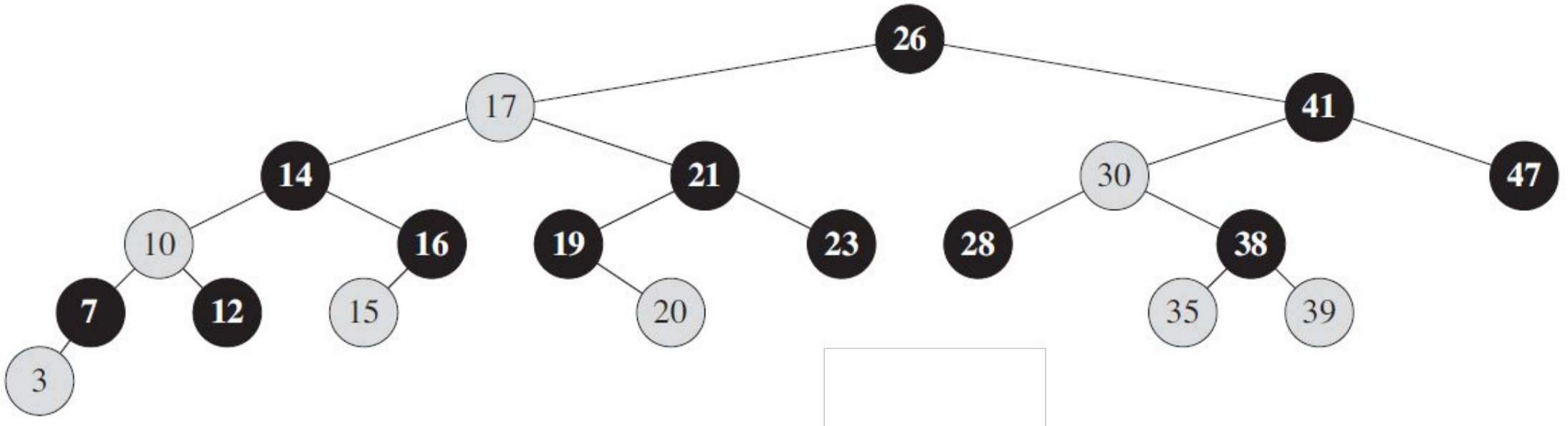
1. Need to show that internal nodes of subtree rooted at any node x is at least $2^{bh(x)} - 1$.

Proof by induction:

1. Base: if $h(x) = 0$ (i.e. x is a leaf – $T.nil$) then $2^{bh(x)} - 1 = 2^0 - 1 = 0$
 2. Induction step: Each child of node x has a black-height $bh(x)$ or $bh(x) - 1$. By assumption, each subtree rooted by a child has at least $2^{bh(x)-1} - 1$ internal nodes.
 3. Conclusion: The subtree rooted by x has at least $2 * (2^{bh(x)-1} - 1) + 1 = 2^{bh(x)} - 1$ internal nodes. $n \geq 2^{bh(x)} - 1$;
2. $h/2 \leq bh(T.root) \Rightarrow 2^{h/2} - 1 \leq n \Rightarrow h \leq 2 \log(n + 1)$

Red-Black Tree: Exercises 1

1. True or False: A Red-Black Tree can be perfectly balanced.
2. What is the largest possible number of internal nodes in a red-black tree with black-height k ? What is the smallest possible number?
3. Add 36 to the following tree. Examine 2 cases: inserted node is red and black



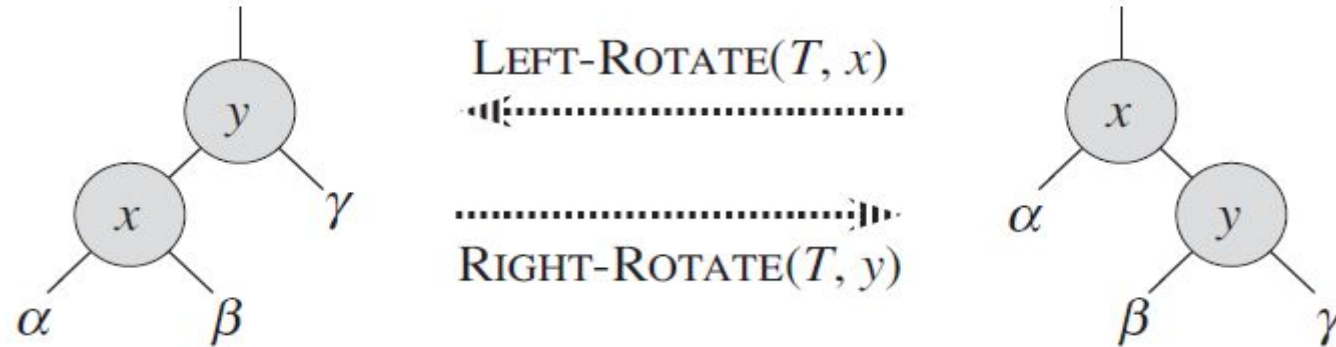
Red-Black Tree: Rotations

Because *TREE-INSERT* and *TREE-DELETE* modify the tree, the result may violate the red-black properties.

To restore these properties, we must change

1. the colors of some of the nodes in the tree
2. the pointer structure (rotate).

Rotation operation change the pointer structure that preserves the binary-search-tree invariants.

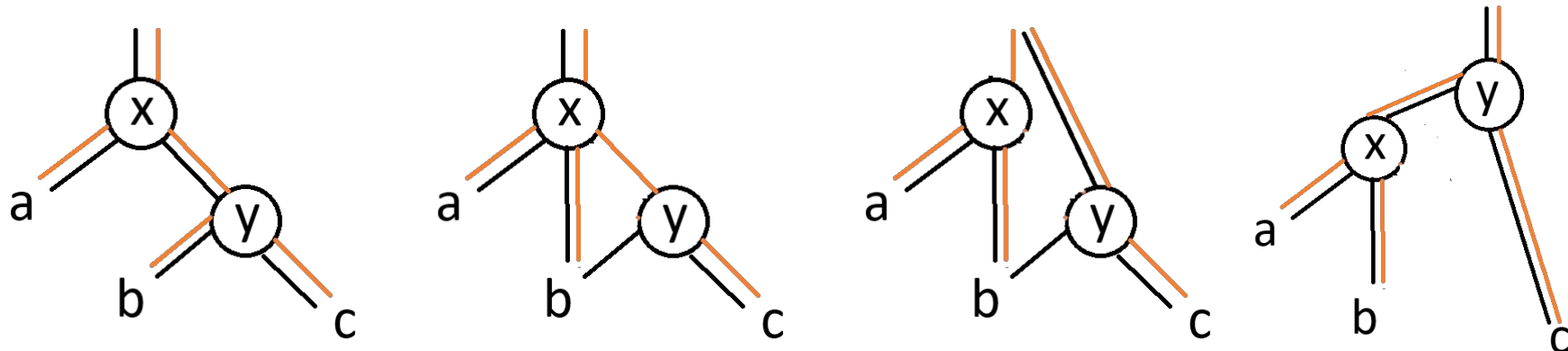
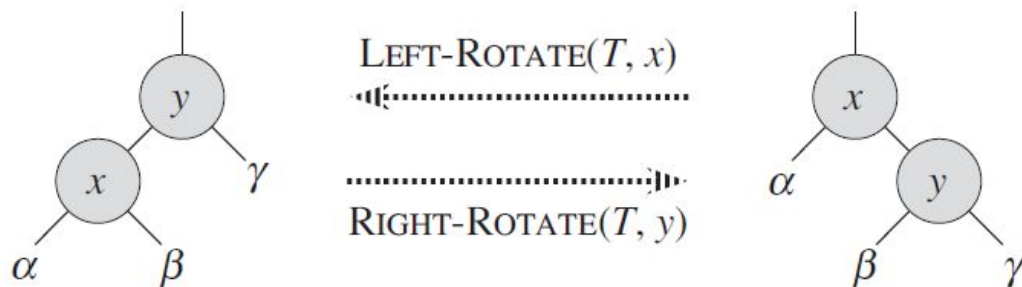


Red-Black Tree: Left Rotate

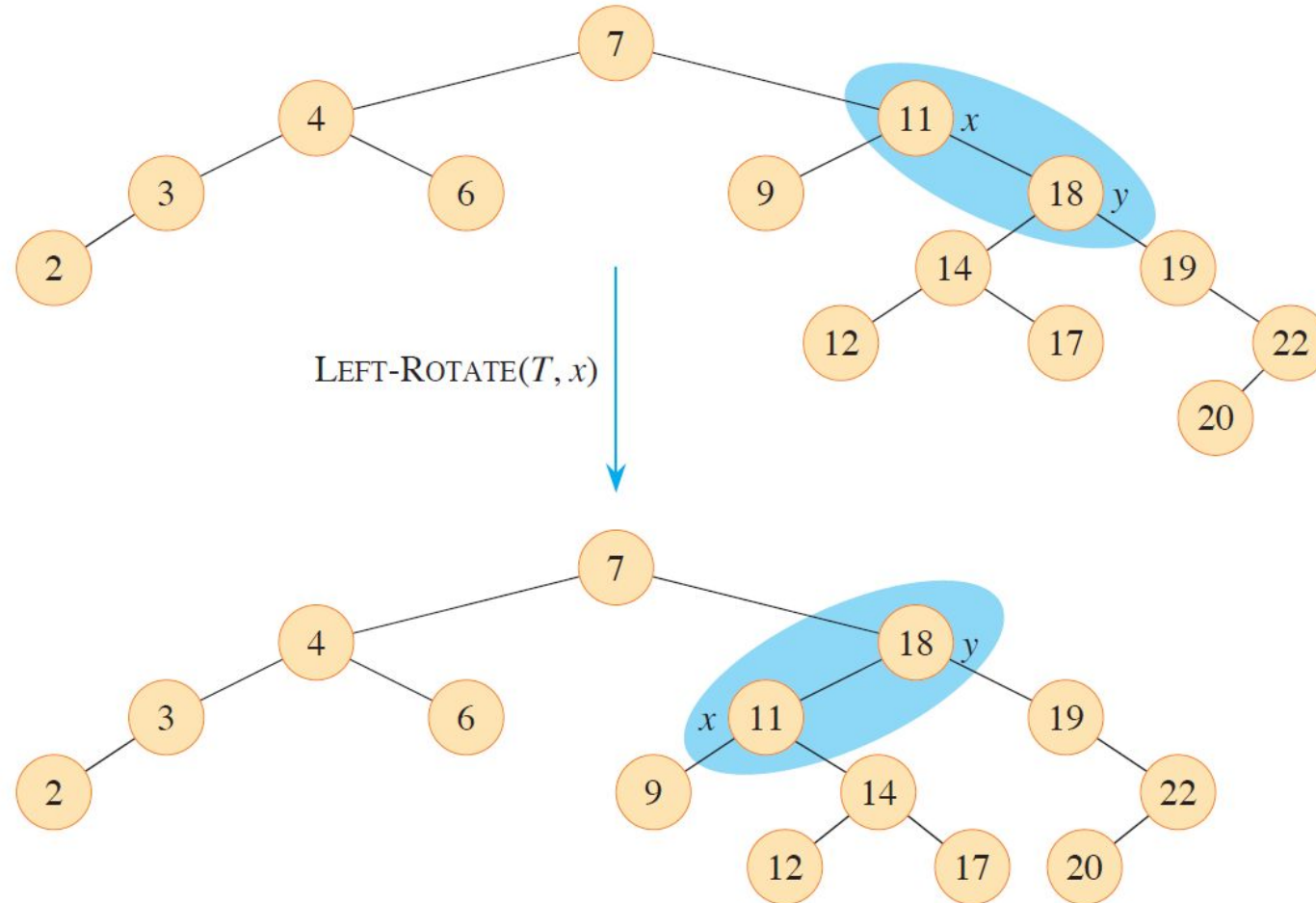
LEFT-ROTATE(T, x)

```

1   $y = x.right$ 
2   $x.right = y.left$ 
3  if  $y.left \neq T.nil$ 
4       $y.left.p = x$ 
5   $y.p = x.p$ 
6  if  $x.p == T.nil$ 
7       $T.root = y$ 
8  elseif  $x == x.p.left$ 
9       $x.p.left = y$ 
10 else  $x.p.right = y$ 
11  $y.left = x$ 
12  $x.p = y$ 
    
```



Red-Black Tree: Left Rotate Example



Red-Black Tree: Exercises 2

1. Write pseudocode for *RIGHT – ROTATE*.
2. How many rotations are possible in a n -node binary search tree?
3. We say that a binary search tree T_1 can be right-converted to binary search tree T_2 if it is possible to obtain T_2 from T_1 via a series of calls to *RIGHT – ROTATE*. Give an example of two trees T_1 and T_2 such that T_1 cannot be right-converted to T_2 . Then, show that if a tree T_1 can be right-converted to T_2 , it can be right-converted using $O(n^2)$ calls to *RIGHT – ROTATE*.

Red-Black Tree: Insert (1)

RB-INSERT(T, z)

```
1   $y = T.nil$ 
2   $x = T.root$ 
3  while  $x \neq T.nil$ 
4       $y = x$ 
5      if  $z.key < x.key$ 
6           $x = x.left$ 
7      else  $x = x.right$ 
8   $z.p = y$ 
9  if  $y == T.nil$ 
10      $T.root = z$ 
11  elseif  $z.key < y.key$ 
12      $y.left = z$ 
13  else  $y.right = z$ 
14   $z.left = T.nil$ 
15   $z.right = T.nil$ 
16   $z.color = RED$ 
17  RB-INSERT-FIXUP( $T, z$ )
```

TREE-INSERT(T, z)

```
1   $y = NIL$ 
2   $x = T.root$ 
3  while  $x \neq NIL$ 
4       $y = x$ 
5      if  $z.key < x.key$ 
6           $x = x.left$ 
7      else  $x = x.right$ 
8   $z.p = y$ 
9  if  $y == NIL$ 
10      $T.root = z$ 
11  elseif  $z.key < y.key$ 
12      $y.left = z$ 
13  else  $y.right = z$ 
```

Red-Black Tree: Insert (2)

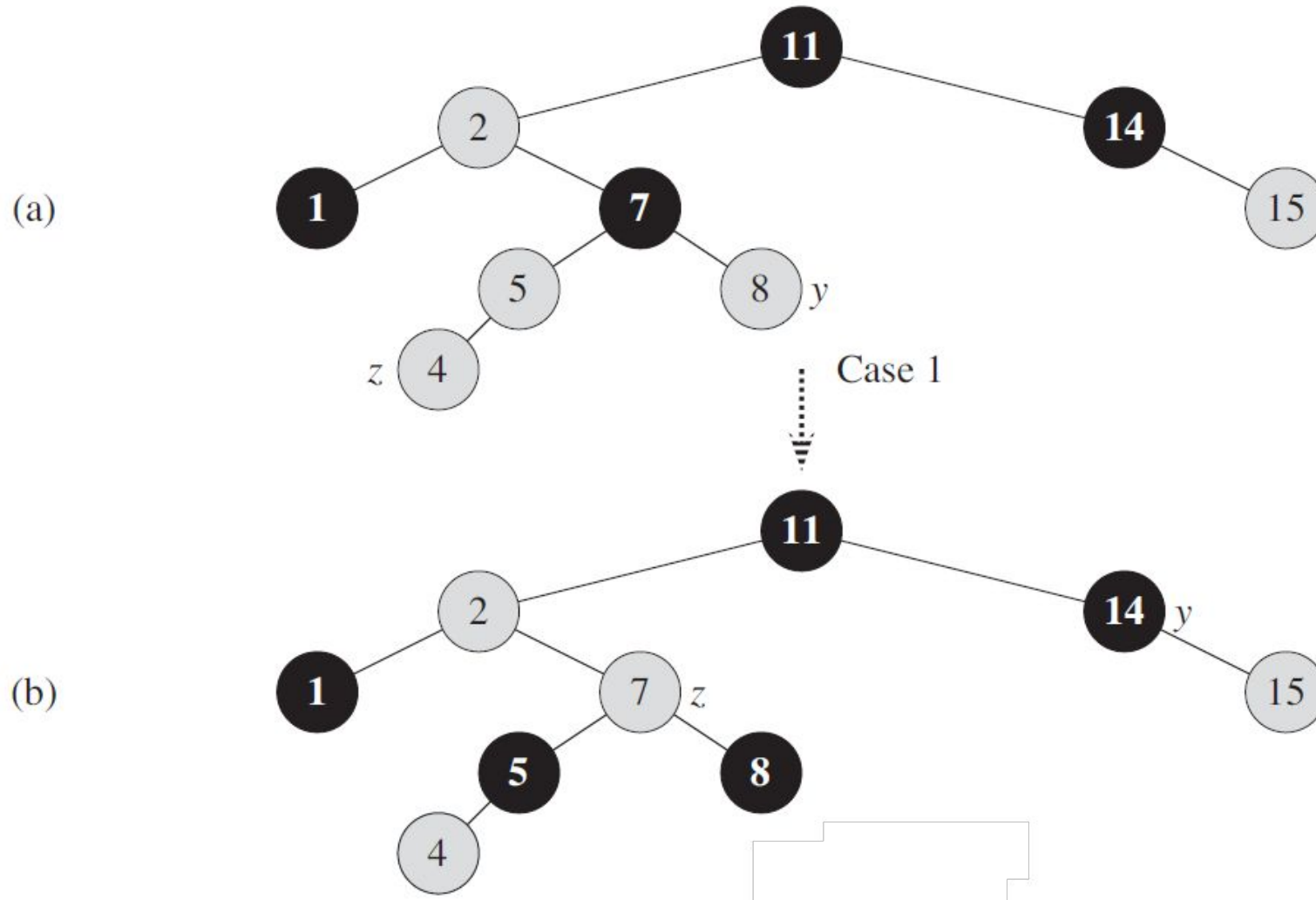
RB-INSERT(T, z)

```
1   $y = T.nil$ 
2   $x = T.root$ 
3  while  $x \neq T.nil$ 
4       $y = x$ 
5      if  $z.key < x.key$ 
6           $x = x.left$ 
7      else  $x = x.right$ 
8   $z.p = y$ 
9  if  $y == T.nil$ 
10      $T.root = z$ 
11  elseif  $z.key < y.key$ 
12      $y.left = z$ 
13  else  $y.right = z$ 
14   $z.left = T.nil$ 
15   $z.right = T.nil$ 
16   $z.color = RED$ 
17  RB-INSERT-FIXUP( $T, z$ )
```

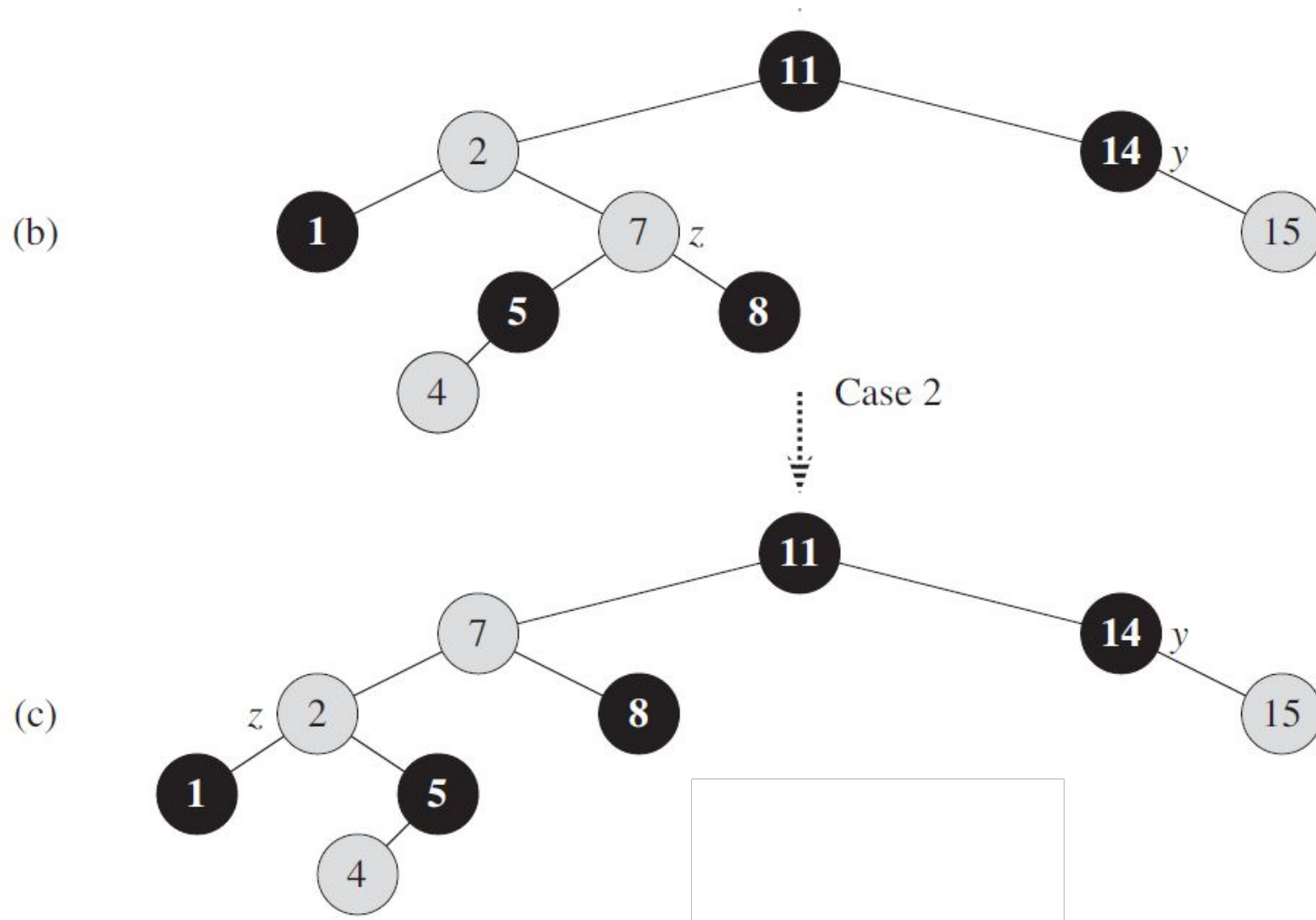
RB-INSERT-FIXUP(T, z)

```
1  while  $z.p.color == RED$ 
2      if  $z.p == z.p.p.left$ 
3           $y = z.p.p.right$ 
4          if  $y.color == RED$ 
5               $z.p.color = BLACK$ 
6               $y.color = BLACK$ 
7               $z.p.p.color = RED$ 
8               $z = z.p.p$ 
9          else
10             if  $z == z.p.right$ 
11                  $z = z.p$ 
12                 LEFT-ROTATE( $T, z$ )
13                  $z.p.color = BLACK$ 
14                  $z.p.p.color = RED$ 
15                 RIGHT-ROTATE( $T, z.p.p$ )
16             else (same as then clause
17                 with “right” and “left” exchanged)
18                  $T.root.color = BLACK$ 
```

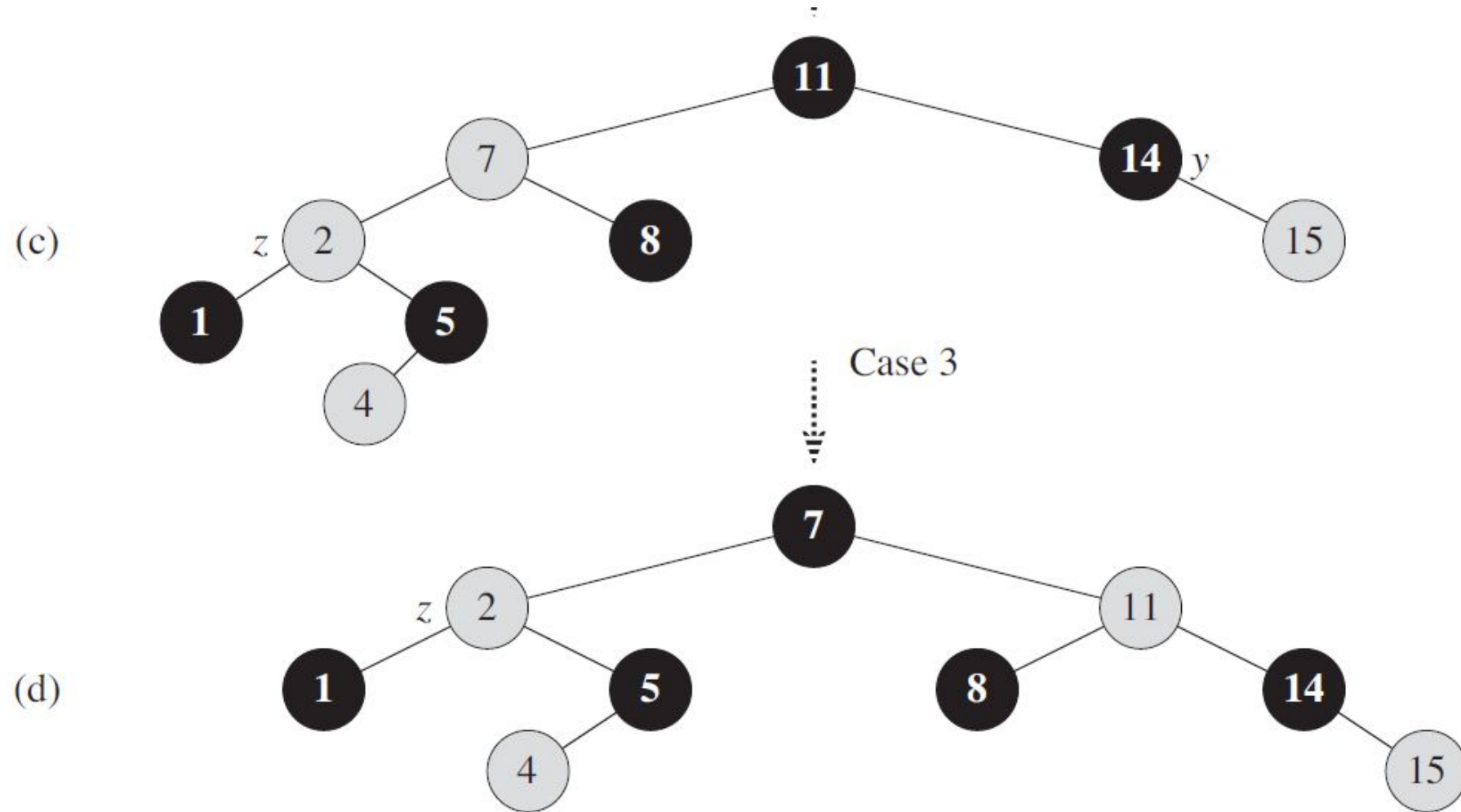

Red-Black Tree: Insert Fixup (1)



Red-Black Tree: Insert Fixup (2)



Red-Black Tree: Insert Fixup (3)



Red-Black Tree: Analyzing Insert Fixup (1)

RB-INSERT-FIXUP(T, z)

```
1  while  $z.p.color == RED$ 
2      if  $z.p == z.p.p.left$ 
3           $y = z.p.p.right$ 
4          if  $y.color == RED$ 
5               $z.p.color = BLACK$ 
6               $y.color = BLACK$ 
7               $z.p.p.color = RED$ 
8               $z = z.p.p$ 
9          else
10             if  $z == z.p.right$ 
11                  $z = z.p$ 
12                 LEFT-ROTATE( $T, z$ )
13                  $z.p.color = BLACK$ 
14                  $z.p.p.color = RED$ 
15                 RIGHT-ROTATE( $T, z.p.p$ )
16             else (same as then clause
17                 with “right” and “left” exchanged)
18          $T.root.color = BLACK$ 
```

The while loop invariant:

- a) node z is red,
- b) If $z.p$ is the root, then $z.p$ is black,
- c) The tree at any moment may violate at most 1 red-black tree property.**

We start with the initialization and termination arguments. Then we shall argue that the loop maintains the invariant upon each iteration. We shall also demonstrate that each iteration of the loop has two possible outcomes: either the pointer z moves up the tree, or we perform some rotations and then the loop terminates.

Red-Black Tree: Analyzing Insert Fixup (2)

RB-INSERT-FIXUP(T, z)

```
1  while  $z.p.color == RED$ 
2      if  $z.p == z.p.p.left$ 
3           $y = z.p.p.right$ 
4          if  $y.color == RED$ 
5               $z.p.color = BLACK$ 
6               $y.color = BLACK$ 
7               $z.p.p.color = RED$ 
8               $z = z.p.p$ 
9          else
10             if  $z == z.p.right$ 
11                  $z = z.p$ 
12                 LEFT-ROTATE( $T, z$ )
13                  $z.p.color = BLACK$ 
14                  $z.p.p.color = RED$ 
15                 RIGHT-ROTATE( $T, z.p.p$ )
16             else (same as then clause
17                 with “right” and “left” exchanged)
18          $T.root.color = BLACK$ 
```

Initialization: Prior to the first iteration of the loop, we started with a red-black tree with no violations, and we added a red node z .

- a) node z is red,
- b) If $z.p$ is the root, then $z.p$ is black,
- c) The tree at any moment may violate at most 1 red-black tree property.

Red-Black Tree: Analyzing Insert Fixup (3)

RB-INSERT-FIXUP(T, z)

```
1  while  $z.p.color == RED$ 
2      if  $z.p == z.p.p.left$ 
3           $y = z.p.p.right$ 
4          if  $y.color == RED$ 
5               $z.p.color = BLACK$ 
6               $y.color = BLACK$ 
7               $z.p.p.color = RED$ 
8               $z = z.p.p$ 
9          else
10             if  $z == z.p.right$ 
11                  $z = z.p$ 
12                 LEFT-ROTATE( $T, z$ )
13                  $z.p.color = BLACK$ 
14                  $z.p.p.color = RED$ 
15                 RIGHT-ROTATE( $T, z.p.p$ )
16             else (same as then clause
17                 with “right” and “left” exchanged)
18          $T.root.color = BLACK$ 
```

Termination:

When the loop terminates, it does so because $z.p$ is black. Thus, the tree does not violate the *property₄* at loop termination.

By the loop invariant, the only property that might fail to hold is *property₂*. Line 17 restores this property.

Red-Black Tree: Analyzing Insert Fixup (4)

RB-INSERT-FIXUP(T, z)

```
1  while  $z.p.color == RED$ 
2      if  $z.p == z.p.p.left$ 
3           $y = z.p.p.right$ 
4          if  $y.color == RED$ 
5               $z.p.color = BLACK$ 
6               $y.color = BLACK$ 
7               $z.p.p.color = RED$ 
8               $z = z.p.p$ 
9          else
10             if  $z == z.p.right$ 
11                  $z = z.p$ 
12                 LEFT-ROTATE( $T, z$ )
13                  $z.p.color = BLACK$ 
14                  $z.p.p.color = RED$ 
15                 RIGHT-ROTATE( $T, z.p.p$ )
16             else (same as then clause
17                 with “right” and “left” exchanged)
18          $T.root.color = BLACK$ 
```

Maintenance:

- There are 6 cases in the while loop, and 3 of them are symmetric to other 3 depending on *line_2*.
- Given the code only for the situation in which $z.p$ is a left child.
- By *part_b* of the invariant the $z.p.p$ node exists.
- We distinguish case_1 from case_2 and case_3 by the color of z 's “uncle.”
- $z.p.p$ is black inside the while loop as $z.p$ is red

Red-Black Tree: Analyzing Insert Fixup (4)

RB-INSERT-FIXUP(T, z)

```
1  while  $z.p.color == RED$ 
2      if  $z.p == z.p.p.left$ 
3           $y = z.p.p.right$ 
4          if  $y.color == RED$ 
5               $z.p.color = BLACK$ 
6               $y.color = BLACK$ 
7               $z.p.p.color = RED$ 
8               $z = z.p.p$ 
9          else
10             if  $z == z.p.right$ 
11                  $z = z.p$ 
12                 LEFT-ROTATE( $T, z$ )
13                  $z.p.color = BLACK$ 
14                  $z.p.p.color = RED$ 
15                 RIGHT-ROTATE( $T, z.p.p$ )
16             else (same as then clause
17                 with “right” and “left” exchanged)
18          $T.root.color = BLACK$ 
```

Maintenance: Case_1 – z 's uncle is red. (lines 5-8)

$$z.p.color = y.color = RED$$

Because $z.p.p$ is black, we can

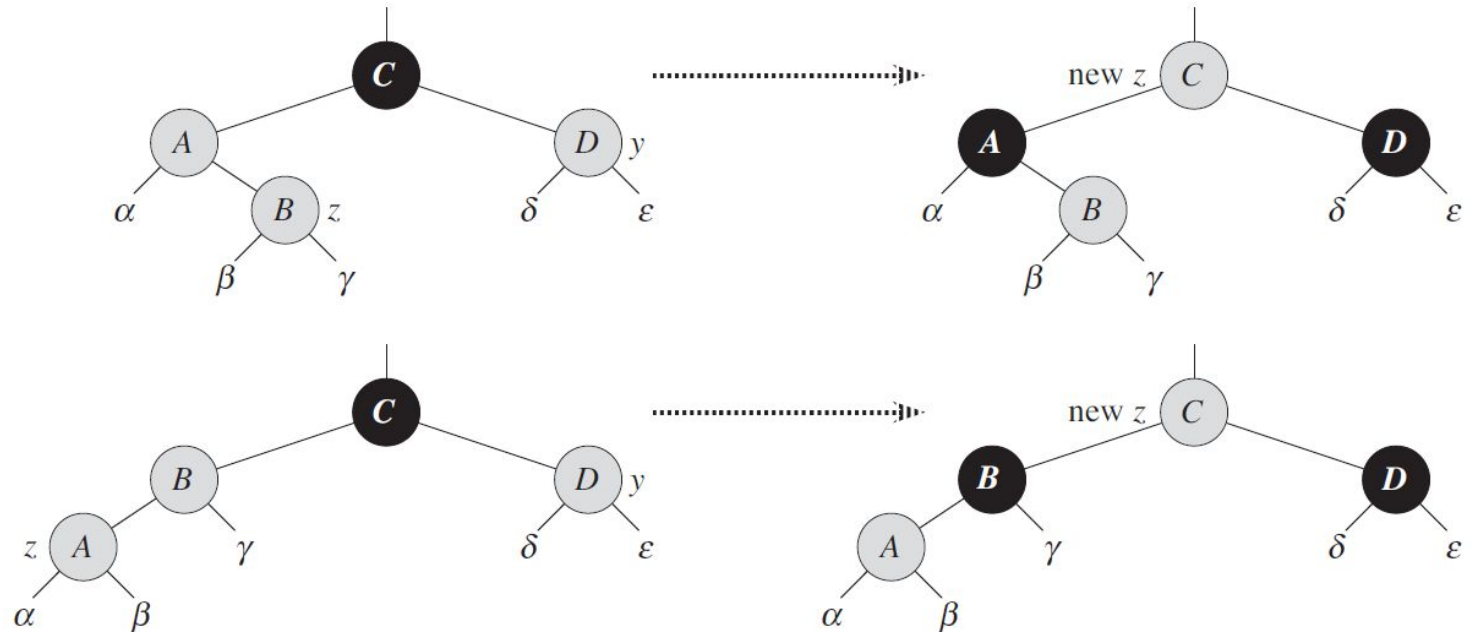
1. color both $z.p$ and y black, thereby fixing the problem of y and $z.p$ both being red,
2. color $z.p.p$ red, thereby maintaining *property_5*.

Red-Black Tree: Analyzing Insert Fixup (5)

RB-INSERT-FIXUP(T, z)

```
1  while  $z.p.color == RED$ 
2      if  $z.p == z.p.p.left$ 
3           $y = z.p.p.right$ 
4          if  $y.color == RED$ 
5               $z.p.color = BLACK$ 
6               $y.color = BLACK$ 
7               $z.p.p.color = RED$ 
8               $z = z.p.p$ 
9      else
10         if  $z == z.p.right$ 
11              $z = z.p$ 
12             LEFT-ROTATE( $T, z$ )
13              $z.p.color = BLACK$ 
14              $z.p.p.color = RED$ 
15             RIGHT-ROTATE( $T, z.p.p$ )
16         else (same as then clause
17             with “right” and “left” exchanged)
18      $T.root.color = BLACK$ 
```

Maintenance: Case_1 – z 's uncle is red. (lines 5-8)
 $z.p.color = y.color = RED$



Red-Black Tree: Analyzing Insert Fixup (6)

RB-INSERT-FIXUP(T, z)

```
1  while  $z.p.color == RED$ 
2      if  $z.p == z.p.p.left$ 
3           $y = z.p.p.right$ 
4          if  $y.color == RED$ 
5               $z.p.color = BLACK$ 
6               $y.color = BLACK$ 
7               $z.p.p.color = RED$ 
8               $z = z.p.p$ 
9          else
10             if  $z == z.p.right$ 
11                  $z = z.p$ 
12                 LEFT-ROTATE( $T, z$ )
13                  $z.p.color = BLACK$ 
14                  $z.p.p.color = RED$ 
15                 RIGHT-ROTATE( $T, z.p.p$ )
16             else (same as then clause
17                 with “right” and “left” exchanged)
18          $T.root.color = BLACK$ 
```

Maintenance:

Case_2 – z 's uncle is black and z is a right child (lines 10-11)

Case_3 – z 's uncle is black and z is a left child (lines 12-14)

$Case_2 + LEFT_ROTATE = Case_3$

Lines 10-12:

- As z and $z.p$ are red, the rotation affects neither the black-height of nodes nor *property*₅.
- $z.p.p$ exists and remains unchanged

Lines 13-14:

- preserves *property*₅
- while loop interrupts in next iteration as $z.p$ is black

Red-Black Tree: Analyzing Insert Fixup (7)

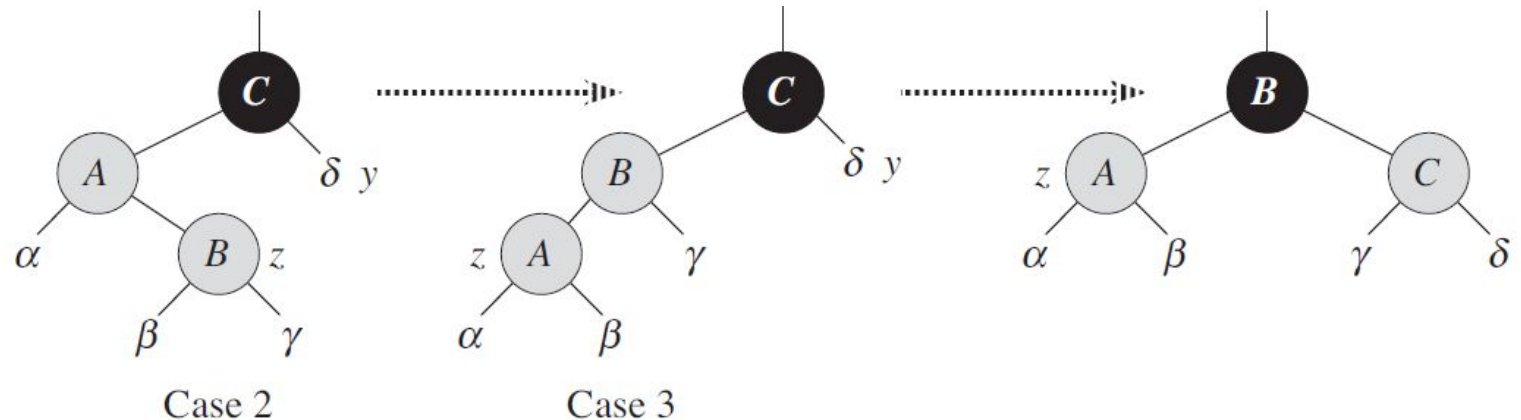
RB-INSERT-FIXUP(T, z)

```
1  while  $z.p.color == RED$ 
2      if  $z.p == z.p.p.left$ 
3           $y = z.p.p.right$ 
4          if  $y.color == RED$ 
5               $z.p.color = BLACK$ 
6               $y.color = BLACK$ 
7               $z.p.p.color = RED$ 
8               $z = z.p.p$ 
9      else
10         if  $z == z.p.right$ 
11              $z = z.p$ 
12             LEFT-ROTATE( $T, z$ )
13              $z.p.color = BLACK$ 
14              $z.p.p.color = RED$ 
15             RIGHT-ROTATE( $T, z.p.p$ )
16         else (same as then clause
17             with “right” and “left” exchanged)
```

Maintenance:

Case_2 – z 's uncle is black and z is a right child (lines 10-11)

Case_3 – z 's uncle is black and z is a left child (lines 12-14)



Red-Black Tree: Insert Fixup (8)

RB-INSERT-FIXUP(T, z)

```
1  while  $z.p.color == RED$ 
2      if  $z.p == z.p.p.left$ 
3           $y = z.p.p.right$ 
4          if  $y.color == RED$ 
5               $z.p.color = BLACK$ 
6               $y.color = BLACK$ 
7               $z.p.p.color = RED$ 
8               $z = z.p.p$ 
9      else
10         if  $z == z.p.right$ 
11              $z = z.p$ 
12             LEFT-ROTATE( $T, z$ )
13              $z.p.color = BLACK$ 
14              $z.p.p.color = RED$ 
15             RIGHT-ROTATE( $T, z.p.p$ )
16  else // same* as lines 3–15
```

```
16      else // same* as lines 3–15
17           $y = z.p.p.left$ 
18          if  $y.color == RED$ 
19               $z.p.color = BLACK$ 
20               $y.color = BLACK$ 
21               $z.p.p.color = RED$ 
22               $z = z.p.p$ 
23          else
24              if  $z == z.p.left$ 
25                   $z = z.p$ 
26                  RIGHT-ROTATE( $T, z$ )
27                   $z.p.color = BLACK$ 
28                   $z.p.p.color = RED$ 
29                  LEFT-ROTATE( $T, z.p.p$ )
30   $T.root.color = BLACK$ 
```

Red-Black Tree: Exercises 2

1. Show the red-black trees that result after successively inserting the keys 41; 38; 31; 12; 19; 8 into an initially empty red-black tree.
2. Show the red-black trees that result after successively inserting the keys 20, 15, 25, 10, 18, 8, 5 into an initially empty red-black tree.
3. Consider a red-black tree formed by inserting n nodes with RB-INSERT. Argue that if $n > 1$, the tree has at least one red node.

Red-Black Tree: Delete: Transplant

RB-TRANSPLANT(T, u, v)

```
1  if  $u.p == T.nil$   
2       $T.root = v$   
3  elseif  $u == u.p.left$   
4       $u.p.left = v$   
5  else  $u.p.right = v$   
6   $v.p = u.p$ 
```

TRANSPLANT(T, u, v)

```
1  if  $u.p == NIL$   
2       $T.root = v$   
3  elseif  $u == u.p.left$   
4       $u.p.left = v$   
5  else  $u.p.right = v$   
6  if  $v \neq NIL$   
7       $v.p = u.p$ 
```


Red-Black Tree: Delete: Transplant

RB-TRANSPLANT(T, u, v)

```
1  if  $u.p == T.nil$   
2       $T.root = v$   
3  elseif  $u == u.p.left$   
4       $u.p.left = v$   
5  else  $u.p.right = v$   
6   $v.p = u.p$ 
```

TRANSPLANT(T, u, v)

```
1  if  $u.p == NIL$   
2       $T.root = v$   
3  elseif  $u == u.p.left$   
4       $u.p.left = v$   
5  else  $u.p.right = v$   
6  if  $v \neq NIL$   
7       $v.p = u.p$ 
```

Red-Black Tree: Delete (1)

RB-DELETE(T, z)

```
1   $y = z$ 
2   $y\text{-original-color} = y.\text{color}$ 
3  if  $z.\text{left} == T.\text{nil}$ 
4       $x = z.\text{right}$ 
5      RB-TRANSPLANT( $T, z, z.\text{right}$ )
6  elseif  $z.\text{right} == T.\text{nil}$ 
7       $x = z.\text{left}$ 
8      RB-TRANSPLANT( $T, z, z.\text{left}$ )
9  else  $y = \text{TREE-MINIMUM}(z.\text{right})$ 
10      $y\text{-original-color} = y.\text{color}$ 
11      $x = y.\text{right}$ 
12     if  $y.p == z$ 
13          $x.p = y$ 
14     else RB-TRANSPLANT( $T, y, y.\text{right}$ )
15          $y.\text{right} = z.\text{right}$ 
16          $y.\text{right}.p = y$ 
17     RB-TRANSPLANT( $T, z, y$ )
18      $y.\text{left} = z.\text{left}$ 
19      $y.\text{left}.p = y$ 
20      $y.\text{color} = z.\text{color}$ 
21 if  $y\text{-original-color} == \text{BLACK}$ 
22     RB-DELETE-FIXUP( $T, x$ )
```

TREE-DELETE(T, z)

```
1  if  $z.\text{left} == \text{NIL}$ 
2      TRANSPLANT( $T, z, z.\text{right}$ )
3  elseif  $z.\text{right} == \text{NIL}$ 
4      TRANSPLANT( $T, z, z.\text{left}$ )
5  else  $y = \text{TREE-MINIMUM}(z.\text{right})$ 
6      if  $y.p \neq z$ 
7          TRANSPLANT( $T, y, y.\text{right}$ )
8           $y.\text{right} = z.\text{right}$ 
9           $y.\text{right}.p = y$ 
10     TRANSPLANT( $T, z, y$ )
11      $y.\text{left} = z.\text{left}$ 
12      $y.\text{left}.p = y$ 
```

Red-Black Tree: Delete (2)

RB-DELETE(T, z)

```
1   $y = z$ 
2   $y\_original\_color = y.color$ 
3  if  $z.left == T.nil$ 
4       $x = z.right$ 
5      RB-TRANSPLANT( $T, z, z.right$ )
6  elseif  $z.right == T.nil$ 
7       $x = z.left$ 
8      RB-TRANSPLANT( $T, z, z.left$ )
9  else  $y = TREE-MINIMUM(z.right)$ 
10      $y\_original\_color = y.color$ 
11      $x = y.right$ 
12     if  $y.p == z$ 
13          $x.p = y$ 
14     else RB-TRANSPLANT( $T, y, y.right$ )
15          $y.right = z.right$ 
16          $y.right.p = y$ 
17     RB-TRANSPLANT( $T, z, y$ )
18      $y.left = z.left$ 
19      $y.left.p = y$ 
20      $y.color = z.color$ 
21 if  $y\_original\_color == BLACK$ 
22     RB-DELETE-FIXUP( $T, x$ )
```

y – the node either removed from the tree or moved within the tree.

$y_original_color$ – stores the color of y before any change

x – node points into node y 's original position.

$x.p$ – points to the original position in the tree of y 's parent

No need to fixup if node y was red as

1. No black-heights in the tree have changed.
2. No red nodes have been made adjacent as y takes z 's position with the z 's color. If y was not z 's right child, then y 's original right child (which is black) replaces y in the tree.
3. Since y could not have been the root if it was red, the root remains black.

Red-Black Tree: Delete (3)

RB-DELETE(T, z)

```
1   $y = z$ 
2   $y\text{-original-color} = y.\text{color}$ 
3  if  $z.\text{left} == T.\text{nil}$ 
4       $x = z.\text{right}$ 
5      RB-TRANSPLANT( $T, z, z.\text{right}$ )
6  elseif  $z.\text{right} == T.\text{nil}$ 
7       $x = z.\text{left}$ 
8      RB-TRANSPLANT( $T, z, z.\text{left}$ )
9  else  $y = \text{TREE-MINIMUM}(z.\text{right})$ 
10      $y\text{-original-color} = y.\text{color}$ 
11      $x = y.\text{right}$ 
12     if  $y.p == z$ 
13          $x.p = y$ 
14     else RB-TRANSPLANT( $T, y, y.\text{right}$ )
15          $y.\text{right} = z.\text{right}$ 
16          $y.\text{right}.p = y$ 
17     RB-TRANSPLANT( $T, z, y$ )
18      $y.\text{left} = z.\text{left}$ 
19      $y.\text{left}.p = y$ 
20      $y.\text{color} = z.\text{color}$ 
21 if  $y\text{-original-color} == \text{BLACK}$ 
22     RB-DELETE-FIXUP( $T, x$ )
```

RB-DELETE-FIXUP(T, x)

```
1  while  $x \neq T.\text{root}$  and  $x.\text{color} == \text{BLACK}$ 
2      if  $x == x.p.\text{left}$ 
3           $w = x.p.\text{right}$ 
4          if  $w.\text{color} == \text{RED}$ 
5               $w.\text{color} = \text{BLACK}$ 
6               $x.p.\text{color} = \text{RED}$ 
7              LEFT-ROTATE( $T, x.p$ )
8               $w = x.p.\text{right}$ 
9          if  $w.\text{left}.color == \text{BLACK}$  and  $w.\text{right}.color == \text{BLACK}$ 
10               $w.\text{color} = \text{RED}$ 
11               $x = x.p$ 
12          else if  $w.\text{right}.color == \text{BLACK}$ 
13               $w.\text{left}.color = \text{BLACK}$ 
14               $w.\text{color} = \text{RED}$ 
15              RIGHT-ROTATE( $T, w$ )
16               $w = x.p.\text{right}$ 
17               $w.\text{color} = x.p.\text{color}$ 
18               $x.p.\text{color} = \text{BLACK}$ 
19               $w.\text{right}.color = \text{BLACK}$ 
20              LEFT-ROTATE( $T, x.p$ )
21               $x = T.\text{root}$ 
22          else (same as then clause with “right” and “left” exchanged)
23       $x.\text{color} = \text{BLACK}$ 
```



Thank You